

1. What is Planning?

- Planning is the task of finding a sequence of actions that achieves a goal from an initial state.
- A plan is a sequence of actions, not a path of states.
- Planning is a form of reasoning about actions, often using a model of the world, rather than exploring the state space directly.

Key Idea: A plan tells an agent what to do (a sequence of actions) to achieve its goals.

2. Initial State, Goal State, Actions, and Transition Model

Initial state	Description of the world at the start.
Goal state	Description of the desired world state (goal condition).
Actions	Set of actions the agent can perform.
Transition model	Specifies the resulting state after an action is applied ($\text{Result}(s, a)$).

A planning problem is specified by an initial state, a goal description, a set of actions, and a transition model.

3. Plan

- A plan is a sequence of actions $[a_1, a_2, \dots, a_n]$ that transforms the initial state into a goal state.
- The plan is successful if, when executed, it results in a state that satisfies the goal.
- Plans can be partial or conditional in more advanced settings.
- Plan quality can be measured by plan length (number of actions) or cost.

Example plan: $[\text{Move}(A, B), \text{Move}(C, A), \text{Move}(B, \text{Table})]$

4. Search vs. Planning

Search	Planning
Searches directly in state space.	Reasons about actions to construct a plan.
Expands states using a transition model.	Uses action descriptions (preconditions and effects).
Finds a path of states.	Finds a sequence of actions (plan).
Typically used for explicit, small state spaces.	Can use symbolic representations and more complex models.

5. Classical Planning Assumptions

- Closed world: the world is fully observable and known.
- Deterministic actions: actions have predictable effects.
- Static environment: the world does not change while planning (no exogenous events).
- Discrete time: actions take an instant to complete.
- Single agent: planning for one agent only.
- Fully specified goals: goal is a set of literals that must be true.
- No resources or uncertainty (in classical planning).

6. STRIPS Representation

- STRIPS (Stanford Research Institute Problem Solver) is a simple and widely used planning representation.
- Each action has a name, a list of preconditions, an add list (positive effects), and a delete list (negative effects).
- World states are represented as a set of ground atomic propositions (literals).

STRIPS action schema:

Action(name(parameters))

Preconditions: $\{p_1, p_2, \dots\}$

Add list: $\{q_1, q_2, \dots\}$

Delete list: $\{r_1, r_2, \dots\}$

7. Preconditions and Effects

- An action can be applied in a state only if its preconditions are satisfied.
- The effect of an action is to add certain literals (make them true) and delete certain literals (make them false).
- The transition model is defined by these effects: if preconditions hold in state s , then $\text{Result}(s, a) = (s - \text{Delete}(a)) \cup \text{Add}(a)$.

Example: $\text{Move}(x, y)$

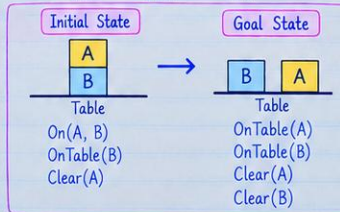
Preconditions: $\{\text{On}(x, z), \text{Clear}(y)\}$

Add list: $\{\text{On}(x, y), \text{Clear}(z)\}$

Delete list: $\{\text{On}(x, z), \text{Clear}(y)\}$

8. Blocks World Example

- A simple domain for illustrating planning.
- Blocks can be on the table or on top of other blocks.
- Actions: $\text{Move}(x, y)$ — move block x onto block y (or onto the table).
- Common predicates: $\text{On}(x, y)$, $\text{OnTable}(x)$, $\text{Clear}(x)$, $\text{Holding}(x)$, ArmEmpty .



9. Why Planning Matters in AI

- Planning allows agents to achieve complex goals by reasoning about the effects of their actions.
- It is essential for real-world AI applications such as robotics, automated planning, logistics, and task scheduling.
- Planning can handle large or infinite state spaces using symbolic representations.
- It enables flexible, goal-directed behavior without enumerating all states.

Examples:

- A robot planning a sequence of moves to assemble an object.
- A delivery agent planning a route of actions to deliver packages.
- An automated system planning a sequence of tasks to achieve a goal.

10. Key Takeaways

- Planning is finding a sequence of actions to achieve a goal.
- A planning problem is defined by an initial state, goal state, actions, and a transition model.
- A plan is a sequence of actions that achieves the goal.
- Planning reasons about actions, unlike search which explores states.
- Classical planning assumes a deterministic, fully observable, static, single-agent environment with fully specified goals.
- STRIPS represents actions using preconditions, add and delete lists.
- An action is applicable only if its preconditions hold, and it changes the state by adding and deleting literals.
- The blocks world is a simple example domain for planning.
- Planning is important for real-world AI applications.
- Planning enables intelligent, goal-directed behavior by reasoning about the effects of actions.

1. Planning Language and Operators

- We describe a problem using a formal language with a set of actions (operators).
- Each operator specifies the conditions for when it can be applied and what it does to the state.
- The language includes a set of predicates, constants, and (optionally) functions.

Operator schema (general form):

Name($a_1, \dots, a_n$)

Preconditions: list of literals that must be true

Add effects: literals that become true

Delete effects: literals that become false

2. STRIPS/PDDL-Style Ideas

- STRIPS uses simple propositional literals with add and delete lists.
- PDDL (Planning Domain Definition Language) adds types, constants, and more expressive constructs (e.g., conditional effects).

Example operator (STRIPS style):

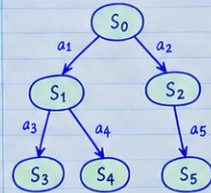
```
Move(x, y)
Pre: At(x), Connected(x, y)
Del: At(y)
Add: At(x)
```

Example operator (PDDL style, typed):

```
(:action move
:parameters (?x - location ?y - location)
:precondition (and (at ?x) (connected ?x ?y))
:effect (and (at ?y) (not (at ?x))))
```

3. State-Space Planning

- Planning can be viewed as a search problem in a state space where states are world descriptions.
- Actions (operators) transform one state into another.
- The goal is to find a sequence of actions (a plan) that leads from the initial state to a goal state.



State-space view:

- Nodes: states
- Edges: actions
- Path: sequence of actions
- Solution: a path to a goal state

4. Forward Progression Planning

- Start at the initial state and repeatedly apply applicable actions to reach the goal.
- Also called forward state-space search.
- At each step, generate successor states by applying actions whose preconditions are true in the current state.
- Can use search algorithms such as BFS, DFS, Uniform-Cost, or heuristic search (e.g., A*).

Key Idea: Build the plan from the initial state towards the goal by applying actions forward.

5. Backward Regression Planning

- Start from the goal description and work backwards to find actions that could achieve it.
- Regress the goal through an operator to get a simpler subgoal (its preconditions).
- Continue until the subgoal matches the initial state.
- Can be more efficient when the goal is specific and there are many irrelevant actions.

Key Idea: Work backward from the goal, replacing a goal with the preconditions of an action that achieves it.

6. Goal Stack Intuition

- Backward planning can be implemented using a stack of goals (subgoals).
- Start with the goal on the stack.
- Pick a goal, choose an action that achieves it, push the action's preconditions, and continue.
- When the stack becomes empty, we have a plan (a sequence of actions).

Goal stack example (idea):

- Stack = [At(Goal)]
- Choose Move(B, Goal) $\rightarrow$ push At(B)
- Choose Move(A, B) $\rightarrow$ push At(A)
- At(A) is true in initial state $\rightarrow$ done.

7. Plan Representation

- A plan is a sequence (or partially ordered set) of actions.
- In classical planning, a plan is usually a linear sequence: $\langle a_1, a_2, \dots, a_n \rangle$.
- Plans can be represented at different levels of detail (e.g., action names with parameters).

Example plan (sequence):

$\pi = \langle \text{Move}(A, B), \text{Move}(B, C), \text{Move}(C, D) \rangle$

8. Example Planning Problem

- Simple navigation problem: move a robot from A to C using actions that move between connected locations.
- Locations: A, B, C
- Actions: Move(x, y) if Connected(x, y)
- Initial state: At(A)
- Goal state: At(C)
- Connections: A-B, B-C

Map:



Operator:

```
Move(x, y)
Pre: At(x), Connected(x, y)
Add: At(y)
Del: At(x)
```

Plan found (forward search):
 $\langle \text{Move}(A, B), \text{Move}(B, C) \rangle$

9. Progression vs. Regression

Aspect	Forward Progression	Backward Regression
Start point	Initial state	Goal state
Search direction	Forward (apply actions)	Backward (use preconditions)
State representation	Complete states	Goal descriptions (partial states)
Branching factor	Often larger	Often smaller (more focused)
When it works well	Good for general goals, plan existence	Good for specific goals, many actions
Typical search	BFS, DFS, A*, etc.	Goal stack, regression search, heuristics
Drawback	May explore many irrelevant states	More complex to implement; may require more reasoning

10. Quick Key Points

- Planning uses a formal language with actions and preconditions/effects.
- STRIPS and PDDL are common planning formalisms.
- Planning is a search problem in a state space.
- Forward progression starts from the initial state and applies actions.
- Backward regression starts from the goal and works through preconditions.
- Goal stack maintains subgoals during backward search.
- A plan is a sequence of actions (or a partial order).
- Progression and regression have different strengths and weaknesses.
- Planning can be solved using general search algorithms and heuristics.
- The choice of planning approach depends on the problem structure and goals.

1. Partial-Order Planning (POP)

- Finds a plan as a partial order of actions instead of a total sequence.
- Specifies only the necessary ordering constraints between actions.
- Leave unrelated actions unordered (more flexibility, less commitment).
- Start with a plan containing only a Start and Finish action, and refine it by adding actions and ordering constraints.

Key Idea: POP searches in the space of partial plans, not complete sequences.

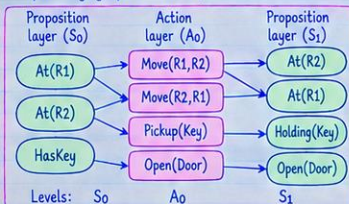
4. Open Preconditions

- A precondition of an action that is not yet supported by any causal link is called open.
- We add a new action (or reuse an existing action) to achieve the open precondition, and add a causal link.

Example: If $\text{Open}(\text{Door})$ requires $\text{Holding}(\text{Key})$, and no action yet achieves $\text{Holding}(\text{Key})$, then $\text{Holding}(\text{Key})$ is an open precondition.

7. Planning Graph Idea

- Build a layered graph of alternating proposition layers and action layers.
- Forward expansion from the initial state using all applicable actions.
- Each proposition at level $k+1$ is produced by some action at level k .
- Helps estimate whether a goal is reachable and provides heuristic information (e.g., planning graph heuristics).



10. Why Partial-Order Planning is Useful

- More flexible: leaves unrelated actions unordered.
- Less commitment: avoids premature decisions and backtracking.
- Can handle independent subgoals naturally (concurrency).

11. Key Takeaways

1. Partial-order planning searches over partial plans.
2. Least-commitment delays unnecessary decisions.
3. Causal links show how preconditions are achieved.
4. Open preconditions need to be supported by new actions.
5. Threats are actions that can make a causal link false.

2. Least-Commitment Planning

- Commit to as little as possible, as late as possible.
- Add ordering constraints only when needed to achieve goals or resolve threats.
- Allows the planner to explore more alternatives and avoid unnecessary choices too early.
- Leads to more flexible and potentially better plans.

Example: Don't decide the order between unrelated actions (e.g., picking up the key and opening the door) until it is necessary.

5. Threats

- An action C threatens a causal link $A \rightarrow B$ if C could make p false between A and B .
- C is a threat if:
 - C has an effect that negates p (i.e., $\neg p$), and
 - C is not ordered before A or after B (so it could occur between A and B).

Example: $C: \text{PutDown}(\text{Key})$ threatens $A \rightarrow \text{Holding}(\text{Key}) \rightarrow B$, because it makes $\neg \text{Holding}(\text{Key})$ true.

8. Mutex Intuition

- Two actions (or propositions) at the same level can be mutex (mutually exclusive) if they cannot both be true/executed.
- Action mutex: actions interfere, have inconsistent effects, or have mutex preconditions.
- Proposition mutex: all ways of achieving them are pairwise mutex.

Examples of action mutex (at same level):

- Inconsistent effects: $\text{Move}(R1,R2)$ mutex $\text{Move}(R2,R1)$
- Interference: $\text{Pickup}(\text{Key})$ mutex $\text{PutDown}(\text{Key})$
- Mutex preconditions: $\text{Open}(\text{Door})$ mutex $\text{Close}(\text{Door})$ (because they need $\neg \text{Open}(\text{Door})$ and $\text{Open}(\text{Door})$).

Examples of proposition mutex (at same level):

- $\text{At}(R1)$ mutex $\text{At}(R2)$ (if only one location is possible)
- $\text{Holding}(\text{Key})$ mutex $\neg \text{Holding}(\text{Key})$
- $\text{Open}(\text{Door})$ mutex $\text{Closed}(\text{Door})$

3. Causal Links

- A causal link shows that an action achieves a condition needed by a later action.
- Notation: $A \rightarrow B$ (A produces p , B needs p)
- A causal link requires that no other action clobbles (makes false) p between A and B .

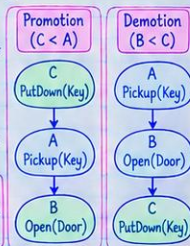
Example: Action $A: \text{Pickup}(\text{Key})$
Action $B: \text{Open}(\text{Door})$
 $\text{Open}(\text{Door})$ requires $\text{Holding}(\text{Key})$.



6. Resolving Threats by Promotion / Demotion

- **Promotion:** add ordering constraint $\text{Holding}(\text{Key}) < A$ (C before A) so the threat cannot occur between A and B .
- **Demotion:** add ordering constraint $B < C$ (C after B) so the threat between A and B .

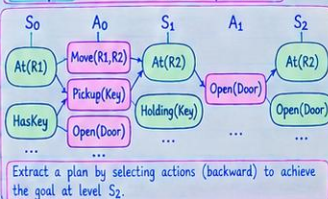
Example:
 $C: \text{PutDown}(\text{Key})$ threatens $A \rightarrow \text{Holding}(\text{Key}) \rightarrow B$.



9. Extraction of a Plan

- A planning graph can be used to extract a plan by backward search from the goal propositions at the last level.
- Choose a consistent set of actions at each level that achieve the needed propositions, avoiding mutex actions.
- Continue backward layer by layer until the initial state is reached.

Example: Goal: $\text{At}(R2) \wedge \text{Open}(\text{Door})$



- Often finds simpler plans than total-order planners.
- Framework for handling threats and causal structure explicitly.
- Forms the basis for many advanced planning algorithms (e.g., constraint-based and hierarchical planning).

6. Threats can be resolved by promotion ($C < A$) or demotion ($B < C$).
7. Planning graphs expand propositions and actions in layers.
8. Mutex relations capture mutual exclusivity between actions or propositions.
9. A plan can be extracted by backward search from a planning graph.
10. Partial-order planning is useful because it is flexible, avoids early commitment, and can find simpler, more general plans.

1. What is Planning?

- Planning is the process of finding a sequence of actions that achieves a goal from an initial state.
- We must consider actions, their preconditions and effects, and the structure of the problem.
- Planning is useful for complex, multi-step tasks in dynamic and uncertain environments.

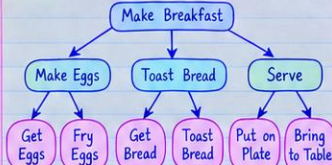
Key Idea: A plan is a sequence of actions that, when executed, leads from the initial state to a goal state.

Example: Making a cup of tea requires multiple steps such as boil water, pour, and add tea.

2. Hierarchical Planning

- Solve complex tasks by decomposing them into smaller subtasks.
- High-level (abstract) tasks are broken down into lower-level subtasks until primitive (executable) actions are reached.
- Reduces search complexity by reasoning at multiple levels of detail.
- Often represented as a task-decomposition hierarchy.

Example: Task-Decomposition Tree



3. Abstraction Levels

- Planning can be performed at different levels of abstraction.
- High-level actions represent complex activities.
- Low-level actions are primitive and directly executable by an agent.
- Higher abstraction reduces search space but requires task decomposition to refine into executable steps.

Example: Making a Sandwich

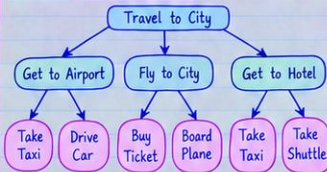
Level	Example Actions
High-level (abstract)	Make Sandwich
Intermediate (subtasks)	Get Ingredients, Prepare Bread, Add Filling, Serve
Low-level (primitive)	Pick Up Bread, Spread Peanut Butter, Place Bread on Plate

Each high-level action is refined into subtasks until we reach primitive actions.

4. High-Level Tasks and Subtasks

- Complex goals are expressed as high-level tasks.
- Each high-level task can be decomposed into subtasks, which may themselves be decomposed.
- This process continues until we reach primitive actions that can be executed.

Example: Travel to a City



5. HTN (Hierarchical Task Network) Idea

- HTN planning starts with a high-level task and uses decomposition methods to break it into subtasks.
- Each method specifies how to decompose a task based on the current state or conditions.
- Decomposition continues until all subtasks are primitive actions.
- HTN is efficient because it uses domain knowledge about how to decompose tasks.

Example: Method for "Travel to City"

Task: Travel-to-City (from, to)

Method 1:

- Get-to-Airport(from)
- Fly(from, to)
- Get-to-Hotel(to)

Methods can have preconditions that determine when a decomposition is applicable.

6. Conditional Planning

- Plans can include conditional actions whose execution depends on the state.
- Allows different actions to be taken based on observations or outcomes.
- Represented using conditional branches or "if-then-else" constructs.

Example: Conditional Plan for Traveling



7. Contingency Plans

- A contingency plan specifies what to do in different situations that may arise.
- Consists of conditional branches to handle unexpected outcomes.
- Useful in dynamic and uncertain environments where the world may not behave as expected.

Example: Contingency Plan for Robot Delivery

- If the door is open → deliver package.
- If the door is closed → ring doorbell and wait.
- If no response → leave package at safe location.
- If package cannot be delivered → return to base.

8. Execution Monitoring

- While executing a plan, the agent monitors the environment to detect whether actions succeed or fail.
- Observations are used to update the agent's belief about the current state.
- If the execution deviates from the plan, the agent may trigger replanning.

Example: Monitoring During Execution

- Execute next action.
- Observe the result.
- If action succeeds, continue with plan.
- If action fails or the state is different than expected, update the state and consider replanning.

9. Replanning

- When the environment changes or actions fail, the agent may need to compute a new plan.
- Replanning uses the updated state information to find a new sequence of actions from the current state.
- Can be combined with execution monitoring for robust behavior.

Example: When to Replan?

- An action fails to achieve its expected effect.
- A new obstacle is detected.
- The goal changes.
- A better plan becomes available.
- The environment is different than expected.

10. Nondeterministic Actions

- Some actions have uncertain or multiple possible outcomes.
- Planning must consider all possible outcomes and ensure the goal can be achieved in every case.
- Often requires conditional or contingent plans and may lead to a plan tree rather than a simple linear sequence.

Example: Nondeterministic Action

Action: OpenDoor()

Possible outcomes:

- Door opens (success)
- Door is locked (failure)

The plan must specify what to do for each possible outcome.

10. Practical Planning Applications

- Robotics: manipulation, navigation, and task execution.
- Automated planning systems: logistics, scheduling, resource management.
- Game playing: planning sequences of moves.
- Personal assistants: plan daily activities.

- Industrial applications: manufacturing, supply chains.
- Planning in uncertain environments: robotics, autonomous.
- Combines domain knowledge, hierarchical decomposition, and replanning for effective real-world behavior.

1. Overall Chapter Summary

- Planning is about finding a sequence (or partially ordered set) of actions that achieves a goal from an initial state.
- We start with classical planning and then extend to more realistic settings (e.g., temporal, uncertain, hierarchical planning).
- Planning algorithms search in the space of states or plans, using operators and heuristics.
- Real-world planning deals with time, resources, uncertainty, and complex goals.

Key Idea: Planning selects and organizes actions to achieve goals efficiently, often under complex real-world constraints.

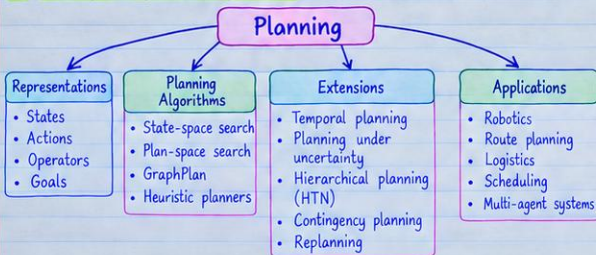
2. Classical vs. Real-World Planning

Classical Planning (idealized)	Real-World Planning (more realistic)
- Deterministic actions (known outcomes) - Fully observable world - Static environment (does not change during planning) - Single agent - Instantaneous actions - Simple goals - Often solvable with state-space search, plan-space search or GraphPlan	- Uncertain or stochastic actions (unknown outcomes) - Partial observability - Dynamic environment (changes while planning or executing) - Multiple agents (multi-agent planning) - Temporal and durative actions (take time) - Complex goals with constraints and preferences - Requires extensions: planning with time, resources, uncertainty, hierarchical planning, replanning

3. Key Terminology Glossary

State	A complete description of the world relevant to the planning problem.
Action	An operation that changes the state. Has preconditions and effects.
Plan	A sequence (or partial order) of actions to achieve a goal.
Operator	A formal description of an action (preconditions and effects).
Causal Link	A step that achieves a precondition for another step in plan-space planning.
HTN	Hierarchical Task Network. Breaks high-level tasks into subtasks.
Contingency Plan	A plan with conditional branches to handle different possible outcomes.
Replanning	Finding a new plan when the world changes or the current plan fails.

4. Concept Map of Chapter 11



5. Common Exam Questions

- Define the planning problem formally (states, actions, goal test, path cost).
- Given a set of operators, find a plan to achieve a goal.
- Compare state-space search and plan-space search.
- Explain GraphPlan and draw the planning graph for a small example.
- Discuss how planning handles uncertainty or partial observability.
- Explain hierarchical planning (HTN) with an example.
- Describe contingency planning and replanning.
- Apply planning to a real-world domain (e.g., robot navigation, logistics, or scheduling).
- Compare classical planning with real-world planning.

6. Common Mistakes

- Forgetting to check action preconditions.
- Confusing states with plans (a plan is a sequence of actions, not a state).
- Ignoring negative effects of actions.
- Not considering that actions can be nondeterministic in real-world settings.
- Assuming the environment is static.
- Overlooking the need for replanning when the world changes.
- Confusing total-order and partial-order plans.

7. Planning Algorithm Comparison

Algorithm	Search Space	Key Idea	Strengths	Limitations
State-space search	States	Search over world states using BFS, DFS, UCS, or heuristic search (e.g., A*)	Simple, well understood	Large state space, ignores plan structure
Plan-space search	Partial plans	Start with partial plan and refine (resolve threats)	Less commitment, flexible	More complex (handling threats)
GraphPlan	Planning graph	Build planning graph and extract plan	Often fast, finds parallel plans	Basic version handles deterministic, propositional domains only
HTN planning	Tasks (hierarchical)	Decompose high-level tasks into subtasks	Scales to complex domains, intuitive	Requires domain-specific task decomposition

8. Real-World Examples

Problem	State Space	Actions	Goal	Notes
Robot tasks	Robot location, objects, status	Move, pick, place, open, close	Complete task (e.g., make coffee)	Requires sequencing and handling constraints
Route planning	Locations, map, traffic conditions	Move between cities/locations	Reach destination	May include time, traffic, and costs
Logistics	Package locations, vehicles, inventory	Load, unload, drive, deliver	Deliver all packages	Involves resources, constraints, and time
Scheduling	Tasks, resources, time slots	Assign task, start, finish	Complete all tasks on time	Temporal planning with resource constraints

“Planning turns goals into action, bridging the gap between what we want and what we do.”

9. Final Key Takeaways

- Planning is finding a sequence or partial order of actions to achieve a goal.
- Classical planning provides the foundation, but real-world planning requires additional models (time, uncertainty, etc.).
- Planning algorithms include state-space search, plan-space search, GraphPlan, and hierarchical planning.
- Key concepts: states, actions, operators, causal links, HTN, contingency plans, and replanning.
- Real-world applications include robotics, route planning, logistics, and scheduling.
- Planning often requires dealing with uncertainty, dynamic environments, and multiple agents.
- Good planning leads to intelligent, autonomous behavior.